

Cylinder vs oriented box

A `UnityEngine.BoxCollider` is only axis-aligned when nothing above it in the hierarchy is rotated, which in practice means ramps, tilted platforms and every yawed wall fall outside `CylinderVsAabb`. The triangle path handles them correctly — a box surface genuinely is 12 triangles — but it is the wrong tool here, for three reasons.

- **Internal-edge ghosts.** A quad split into two triangles has a diagonal seam that is not a real edge. A cylinder sliding across it generates a contact whose normal points along the seam, and the character catches on geometry that is visually flat. This is the classic character-controller bug, and it is entirely an artifact of triangulating.
- **Contact count.** Up to 12 contacts per box, most of them redundant, all of them competing inside the relaxation loop. A direct test emits exactly one.
- **Grounding quality.** `ClassifyGrounding` reads contact normals. One clean face normal classifies a ramp correctly; a fan of triangle normals plus seam normals does not.

So `UnityWorldProbe` should dispatch `BoxCollider` to a dedicated test and leave the triangle path to `MeshCollider`, where it is genuinely needed.

The test below is exact for every axis-aligned box, exact for every box yawed about world Y — which together is most of a hand-built level — and a bounded, safe-direction approximation only for genuinely tilted boxes. The last page says precisely where the bound sits.

Building the box

Take the rotation and the extents separately. The frame must stay **orthonormal**: if scale leaks into the axes, the transform below stops being rigid and the cylinder arrives in box space as an elliptic cylinder, which none of the math that follows is written for.

```
public struct Obb
{
    public Vector3 Center;           // world
    public Vector3 AxisX, AxisY, AxisZ; // orthonormal, rotation only
    public Vector3 HalfExtents;      // world units; scale lives here
}

public static Obb FromUnity(UBoxCollider b)
{
    Transform t = b.transform;
    Quaternion q = t.rotation;
    Vector3 s = t.lossyScale;
    return new Obb {
        Center = t.TransformPoint(b.center),
        AxisX = q * Vector3.right, AxisY = q * Vector3.up, AxisZ = q * Vector3.forward,
        HalfExtents = 0.5f * new Vector3(b.size.x * Mathf.Abs(s.x),
                                         b.size.y * Mathf.Abs(s.y),
                                         b.size.z * Mathf.Abs(s.z)),
    };
}
```

A rotated child under a non-uniformly scaled parent is a sheared box, and neither this struct nor PhysX's own `BoxCollider` can represent shear — Unity approximates it the same way. The approximation is inherited rather than introduced, but it is worth knowing before someone scales a rotated platform on one axis and wonders why the collision sits slightly off the render mesh.

Into box space

Because the frame is orthonormal, moving into it is a rigid motion: the box becomes an AABB at the origin, and the cylinder stays a cylinder, merely tilted.

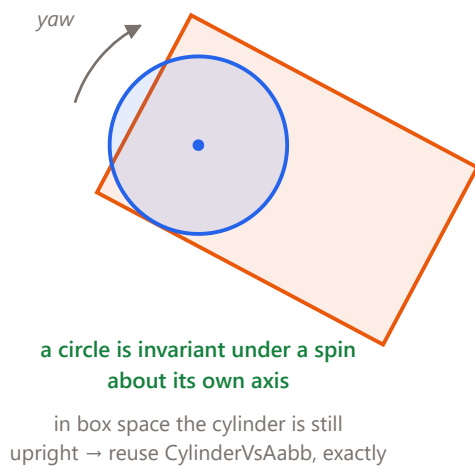
```
Vector3 dW = c.Center - box.Center;
Vector3 p = new Vector3(Vector3.Dot(dW, box.AxisX), // cylinder centre, box space
                        Vector3.Dot(dW, box.AxisY),
                        Vector3.Dot(dW, box.AxisZ));

// World +Y expressed in box space. This is a row of the rotation matrix, not a
// transform: dot(worldY, AxisX) is just AxisX.y. Unit length for free.
Vector3 u = new Vector3(box.AxisX.y, box.AxisY.y, box.AxisZ.y);
```

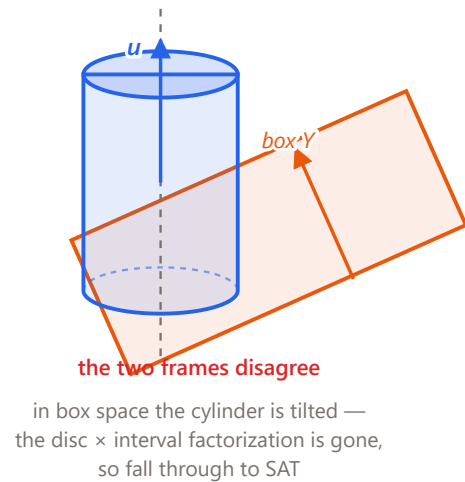
`u` is the whole problem in one vector: it is where the cylinder's axis ends up once the box has been straightened out.

Two tiers, decided by one number

TOP-DOWN · box yawed about world Y



ELEVATION · box tilted about X or Z



A cylinder is a surface of revolution about its own axis, so **rotating the box about world Y changes nothing about the cylinder**. If $|u.y| \approx 1$ then the box's own Y axis is world Y (up to a flip, which an AABB is symmetric under), the cylinder is still upright in box space, and the exact closed-form `CylinderVsAabb` applies unchanged. Only the normal has to be rotated back out.

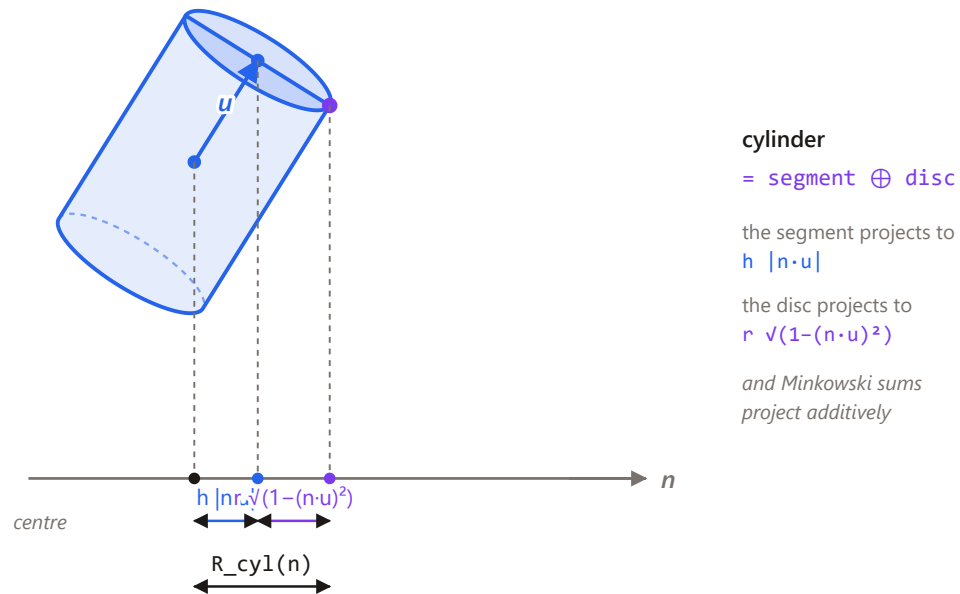
```
const float TILT_EPSILON = 1e-4f; // |u.y| this close to 1 == no tilt

static bool YawOnly(in Cylinder c, in Obb box, Vector3 p, out Contact contact)
{
    Cylinder local = c; local.Center = p;
    Aabb aabb = new Aabb { Center = Vector3.zero, HalfExtents = box.HalfExtents };
    if (!CylinderVsAabb(local, aabb, out contact)) return false;
    contact.Normal = ToWorld(contact.Normal, box);
    contact.Point = box.Center + ToWorld(contact.Point, box);
    return true;
}

static Vector3 ToWorld(in Vector3 v, in Obb b)
    => v.x * b.AxisX + v.y * b.AxisY + v.z * b.AxisZ;
```

This is worth more than it looks. Rotated walls, pillars, doorframes and yawed platforms are the overwhelming majority of non-axis-aligned boxes in a hand-built level, and every one of them keeps the exact test at the cost of one extra transform. Only genuine tilt — a ramp, a fallen slab — reaches the general path.

The projection radius



Once the cylinder is tilted, the disc-times-interval factorization is gone and there is no minimum translation vector to read straight off. What survives is the fact the whole narrowphase is built on: **a cylinder is a segment Minkowski-summed with a disc**, and extents along an axis add under Minkowski sum. So for any unit axis n the cylinder's half-extent stays closed form — the segment contributes $h \cdot |n \cdot u|$, the disc contributes $r \cdot \sqrt{1 - (n \cdot u)^2}$:

```
R_cyl(n) = h * |n · u| + r * √(1 - (n · u)²)    // cylinder, any orientation
R_box(n) = e.x * |n.x| + e.y * |n.y| + e.z * |n.z|    // an AABB in this frame
```

which makes n a separating axis exactly when $|p \cdot n| > R_{\text{box}}(n) + R_{\text{cyl}}(n)$. Both endpoints sanity-check: at $n \cdot u = \pm 1$ the cylinder's extent is h , and at $n \cdot u = 0$ it is r .

Note what is *not* being claimed. The identity is exact for every axis. What follows is a finite *choice* of axes, and that choice is where the approximation enters.

The axis set

Separating-axis testing needs every axis normal to a face of the configuration-space obstacle $\text{box} \oplus \text{cylinder}$. Four families are finite and cheap:

family	axes	what it catches, and how it simplifies
box face normals	X, Y, Z	face contact; R_{box} collapses to one extent
cylinder caps	u	standing on the box; $n \cdot u = 1$, so $R_{\text{cyl}} = h$
box edge $\times$ side wall	$u \times X, u \times Y, u \times Z$	edge vs the curved wall; $n \cdot u = 0$, so $R_{\text{cyl}} = r$
nearest-feature azimuth	$\text{perp}_e(q - p)$	the completion below; also $n \cdot u = 0$

The fourth family is not optional, and the reason is easy to miss. The side wall's normals form a **continuous** family — every direction perpendicular to u — and the first three families sample roughly three azimuths of it. A box sitting at 45° between two box axes then has no sampled axis pointing at it:

Cylinder $r = 1$, $h = 0.5$ at the origin; a small box centred at $(0.742, 0.4, 0.742)$. Its radial distance is $1.049 > r$, so the two are genuinely apart — yet X and Z each see only 0.742 against a wall extent of 1 , u sees 0.4 against a cap extent of 0.5 , and the three cross products merely reproduce X and Z . Seven axes, no separation found, phantom contact. The completion axis separates it by 0.048 .

The fix is one axis: clamp the cylinder's centre to the box to get the nearest point, then take the component of that offset perpendicular to u . That is the azimuth the wall actually needs, and it is the same clamp the AABB test already performs.

```
Vector3 q = new Vector3(Mathf.Clamp(p.x, -e.x, e.x), // nearest point on the box
                        Mathf.Clamp(p.y, -e.y, e.y),
                        Mathf.Clamp(p.z, -e.z, e.z));
Vector3 w = q - p;
Vector3 radial = w - u * Vector3.Dot(w, u);           // strip the axial part
```

One axis, one test

Bundling the invariants into a small struct keeps the axis list on the next page to one line per axis, and keeps it allocation-free — this runs inside the substep loop.

```
struct Sat
{
    public Vector3 P;          // cylinder centre, box space
    public Vector3 E;          // box half extents
    public Vector3 U;          // world +Y in box space
    public float   R, H;
    public float   BestDepth;
    public Vector3 BestAxis;
}

// Returns false the moment this axis separates the pair. Otherwise it keeps the
// running minimum, which on exit is the minimum translation vector.
static bool Test(ref Sat s, Vector3 n)
{
    float lenSq = n.sqrMagnitude;
    if (lenSq < EpsilonSq) return true; // degenerate: carries no information,
    n /= Mathf.Sqrt(lenSq);             // and normalizing it would be pure noise

    float nu = Vector3.Dot(n, s.U);
    float rCyl = s.H * Mathf.Abs(nu) + s.R * Mathf.Sqrt(Mathf.Max(0f, 1f - nu * nu));
    float rBox = s.E.x * Mathf.Abs(n.x) + s.E.y * Mathf.Abs(n.y) + s.E.z * Mathf.Abs(n.z);

    float dist = Vector3.Dot(s.P, n);
    float depth = (rBox + rCyl) - Mathf.Abs(dist);
    if (depth <= 0f) return false;      // separating axis -- disjoint, done

    if (depth < s.BestDepth)
    {
        s.BestDepth = depth;
        s.BestAxis = dist >= 0f ? n : -n; // B toward A, per the Contact convention
    }
    return true;
}
```

The `Mathf.Max(0f, ...)` inside the square root is not decoration. For a near-parallel axis `nu` can land a few ulps past 1, and `Sqrt` of a small negative is `NaN` propagating straight into a contact normal and from there into the solver, where it poisons a body's position permanently.

Assembling the contact

```
public static bool CylinderVsObb(in Cylinder c, in Obb box, out Contact contact)
{
    contact = default;

    Vector3 dW = c.Center - box.Center;
    Vector3 p = new Vector3(Vector3.Dot(dW, box.AxisX),
                           Vector3.Dot(dW, box.AxisY),
                           Vector3.Dot(dW, box.AxisZ));
    Vector3 u = new Vector3(box.AxisX.y, box.AxisY.y, box.AxisZ.y);

    if (1f - Mathf.Abs(u.y) < TILT_EPSILON)
        return YawOnly(c, box, p, out contact);           // the exact path

    Sat s = new Sat { P = p, E = box.HalfExtents, U = u,
                     R = c.Radius, H = c.HalfHeight, BestDepth = float.MaxValue };

    // 1. box face normals
    if (!Test(ref s, Vector3.right) || !Test(ref s, Vector3.up)
        || !Test(ref s, Vector3.forward)) return false;

    // 2. the cylinder's own axis
    if (!Test(ref s, u)) return false;

    // 3. box edge x side-wall generator; u x X is (0, u.z, -u.y), and so on
    if (!Test(ref s, new Vector3(0f, u.z, -u.y)) ||
        !Test(ref s, new Vector3(-u.z, 0f, u.x)) ||
        !Test(ref s, new Vector3(u.y, -u.x, 0f))) return false;

    // 4. the nearest-feature azimuth
    Vector3 q = new Vector3(Mathf.Clamp(p.x, -s.E.x, s.E.x),
                           Mathf.Clamp(p.y, -s.E.y, s.E.y),
                           Mathf.Clamp(p.z, -s.E.z, s.E.z));

    Vector3 w = q - p;
    if (!Test(ref s, w - u * Vector3.Dot(w, u))) return false;

    // Nothing separated them, so BestAxis is the shortest exit we found.
    contact.A = c.Body;
    contact.B = null;                               // Unity world: infinite mass
    contact.Normal = ToWorld(s.BestAxis, box);
    contact.Depth = s.BestDepth;
    contact.Point = box.Center + ToWorld(q, box);    // debug visualisation only
    return true;
}
```

The `||` chains short-circuit, which is exactly right: once one axis separates the pair the remaining axes carry no information and the function is already returning `false`.

What this is exact about, and what it is not

The four families cover every *flat* face of the configuration-space obstacle. What they do not cover are the curved patches it grows where the cylinder's **rim circles** sweep against box vertices and edges. Those carry a continuum of normals, and no finite axis list reaches them.

The consequence is bounded, and bounded in the safe direction. A missing axis can only cause SAT to **fail to find** a separation that exists — never the reverse.

Can happen

- a phantom contact when a cap rim passes very near a tilted box's edge
- a depth slightly larger than the true minimum
- tilted-box edges that feel faintly rounded

Cannot happen

- a missed overlap
- tunnelling
- the character sinking into a box

If it ever becomes visible, the vertex half closes exactly and still without iterating: for each box vertex v , add the axis from v to its nearest point on the near rim circle — and the nearest point on a circle to a point is a plain radial projection, closed form. Eight more axes. Only rim-versus-box-edge resists, because that one is a genuine quartic. It is the single place in Phym where the no-iteration rule would have to bend, and it is not worth bending for a character 0.32 units wide.

Wiring it in

- `UnityWorldProbe.Generate` dispatches `UBoxCollider` here instead of to the 12-triangle path. `MeshCollider` is unchanged.
- Build order step 6 shrinks: boxes no longer wait on `MeshTriangleCache`, so this can land alongside step 4 and give the solver tilted geometry to chew on much earlier.
- The broadphase already did the cheap reject — `Physics.OverlapBoxNonAlloc` handed us this collider — so there is no AABB pre-test to add here.
- `Contact.B == null` keeps the infinite-mass path, and the normal convention is the same as everywhere else: from B toward A.

Tests worth writing

1. Box yawed 45° about Y: assert the fast path is taken, and that the contact matches `CylinderVsAabb` run in box space to within float noise.
2. The counterexample above, as a regression test: assert `CylinderVsOBB` returns `false`. Delete the fourth axis family and watch it fail.
3. Stand on a 20° ramp: one contact, normal equal to the ramp's up axis, `IsGrounded` `true`. Then 60°: still one contact, `IsGrounded` `false`, the character slides.
4. Walk the length of a tilted slab: the normal must stay constant. Any flicker means competing axes are winning intermittently — the failure the triangle path produced by construction.
5. Drop a cylinder onto a tilted box's top edge and let it settle. It must come to rest rather than buzz between the face axis and the edge axis.
6. A box scaled to near zero on one axis: `Test` must reject the degenerate axes rather than emit `NaN`.